MINISTRY OF SCIENCE AND TECHNOLOGY

POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY



PROJECT REPORT

FACE RECOGNITION METHODS

Supervisor : Master. NGUYEN NGOC DUY

Students : NGUYEN CONG ANH TIEN - K23DTCN216

: PHAM THI NGOC THAO - K24DTCN256

: LUONG HONG HUNG - K25DTCN324

Subject : Artificial Intelligence

Ho Chi Minh City, December 2025

Table of contents

Task Assignment ..... *i*

1. *System Overview* *3*
2. *System Architecture* *4*
3. *Data Structure* *5*
4. *AI Algorithms* *6*
5. *API Documentation* *7*
6. *Database* *9*
7. *User Interface* *11*
8. *Appendix* *15*
9. *References* *15*

Task Assignment

Member	Role	Detailed Responsibilities
Pham Thi Ngoc Thao	Backend & AI Engineer	- Develop the API system using FastAPI. - Integrate and train YOLOv8 for face detection. - Deploy FaceNet to extract facial feature embeddings. - Implement image processing algorithms (resizing, NMS).
Nguyen Cong Anh Tien	Frontend Developer (Mobile)	- Design the UI/UX using SwiftUI. - Implement image selection from the photo library and camera on iOS. - Connect backend APIs to display search results and user lists. - Manage application states (loading and error handling).
Luong Hong Hung	Database & DevOps	- Set up and administer MongoDB (user information storage). - Deploy and optimize Qdrant Vector Database for face matching. - Containerize the application with Docker and create Docker Compose configuration. - Write the report and system installation guide.

1. System Overview

1.1. Introduction

The Face Recognition System is an advanced technology solution that uses artificial intelligence to automate user identification and management. It combines modern AI technologies, including computer vision, deep learning, and vector similarity search, to deliver accurate and fast recognition.

1.2. System Objectives

- Automated recognition: Automatically verify user identities, reduce manual operations, and save time.
- High accuracy: Achieve recognition accuracy above 95% under normal lighting conditions.
- Fast processing: Recognition time below 2 seconds per verification.
- Centralized management: Provide centralized management of users and recognition data.

1.3. Key Features

- User management
- Face recognition
- APIs and integration
- Data management

1.4. Overall Architecture

Simplified architecture: Presentation → Business Logic → Data Access → Infrastructure

Microservice Components

- API Gateway: FastAPI server with load balancing
- User Service: Manages user information
- Face Recognition Service: Performs facial recognition
- Image Processing Service: Processes and optimizes images
- Database Services: MongoDB and Qdrant

1.5. Technologies Used

1.5.1. Backend Technologies

- Framework: FastAPI 0.104.1 (Python)
- Web Server: Uvicorn ASGI server
- Authentication: JWT tokens (future implementation)
- Validation: Pydantic models

1.5.2. Frontend Technologies

- Framework: SwiftUI
- IDE: Xcode
- Language: Swift
- Platform: iOS

1.5.3. AI/ML Technologies

- Face Detection: YOLOv8n (You Only Look Once v8 nano)
- Face Recognition: FaceNet (InceptionResnetV1)
- ML Framework: PyTorch (CPU optimized)
- Image Processing: Pillow (PIL)

1.5.4. Database Technologies

- Primary Database: MongoDB 7.0+ (document store)
- Vector Database: Qdrant (similarity search)
- Caching: Redis (future implementation)
- Search Engine: Elasticsearch (future implementation)

1.5.5. Infrastructure Technologies

- Containerization: Docker & Docker Compose
- Orchestration: Kubernetes (production)
- Monitoring: Prometheus + Grafana
- Logging: ELK Stack (Elasticsearch, Logstash, Kibana)

2. System Architecture

The system uses a layered architecture combined with microservices to ensure modularity, scalability, and maintainability. It is optimized for real-time processing and high scalability.

2.1. Overall Architecture

Simplified overall architecture diagram:

- Client (Web/Mobile/API Client) → FastAPI Server
- FastAPI Server → Business services (User Manager, Face Manager, Image Processor)
- Business services → Databases (MongoDB stores user information; Qdrant stores face vectors)
- Deployment infrastructure: Docker Compose manages MongoDB and Qdrant.

2.2. Description of Architecture Layers

2.2.1. Presentation/API Layer

- Provides REST APIs for creating users, retrieving user lists, adding images, and searching by face.
- Automatically generates API documentation (Swagger) and validates input data.

2.2.2. Business Logic Layer

- User Manager: Coordinates user creation/updates, business validation, and data storage.
- Face Manager: Executes the recognition pipeline (detect → encode → search) and calculates similarity.
- Image Processor: Processes images, generates thumbnails, and encodes/decodes base64.

2.2.3. AI/ML Layer

- YOLOv8n: Detects face locations in input images.
- FaceNet: Converts faces into feature vectors (embeddings) for matching.
- Similarity Search: Finds the nearest vectors in Qdrant using cosine similarity and a configured threshold.

2.2.4. Data/Infrastructure Layer

- MongoDB: Stores user information, metadata, images, and thumbnails.
- Qdrant: Stores embeddings and returns similarity search results.
- Docker Compose: Initializes and manages MongoDB and Qdrant containers.

2.3. Main Data Flows

2.3.1. Create User (POST /users)

- The client sends user information and 1–5 face images (base64).
- The API validates the request → User Manager handles business logic.
- Images are preprocessed (resizing/thumbnails) → Face Manager detects and encodes faces.
- Embeddings are stored in Qdrant; user information and images are stored in MongoDB.
- The user creation result is returned.

2.3.2. Search by Face (POST /search)

- The client sends one base64-encoded image.
- The API validates the request → Face Manager detects and encodes the face.
- Qdrant finds the nearest vector. If it meets the threshold, user information is retrieved from MongoDB and returned with a confidence score; otherwise, the response indicates “not found.”

3. Data Structure

3.1. Overview

The face recognition system uses two databases: MongoDB for user data and metadata, and Qdrant for storing and searching facial feature vectors (embeddings).

3.2. MongoDB Design (Simplified)

- Database: The sample documentation uses face_attendance; for a recognition-focused deployment, it can be renamed face_recognition.
- Main collection: users (user information). Additional collections such as audit_logs and system_config can be added for monitoring and system configuration.

3.2.1. Users Collection (Main Fields)

- Basic information: _id, name, email, phone, department, created_at, updated_at, status.
- images: An array of 1–5 entries, each containing image_id, full_image (base64), thumbnail (base64), and face_vector_id (reference to Qdrant).
- metadata: total_images, last_recognition, recognition_count for quick statistics.

3.2.2. Constraints and Data Validation

- Email is required and should have a unique constraint to prevent duplicates.
- Name must contain at least two characters; status must be active, inactive, or deleted.
- Each user may have at most five images; image_id uses UUID format.

3.2.3. Indexes

- Single-field indexes: email (unique), name, department, status, created_at, images.image_id.
- Compound indexes: status+department; email+status.
- Text indexes: name, email, department for fast keyword searches.

3.3. Qdrant Design

- Collection: face_vectors.
- Vector size: 512 (FaceNet output); distance metric: Cosine for similarity matching.
- Each point has an id (UUID), a vector (512 floating-point values), and a payload containing user_id, image_id, created_at, face_bbox, and optional model/quality information.

3.4. Data Relationships

- Each image generates one corresponding face vector (face_vector_id), stored in Qdrant.
- Deleting a user requires removing all images from MongoDB and associated vectors from Qdrant.
- Vector integrity: embeddings must have exactly 512 dimensions and be normalized before storage.

3.5. Storage Optimization

- Full-size images: resize to approximately 600 px wide to balance quality and storage; thumbnails: 150×150 for fast display.
- Store vectors in batches for better performance; consider caching repeated search results.

3.6. Backup and Security

- MongoDB: daily backups using mongodump; Qdrant: daily snapshots retained according to configuration.
- Biometric data (face vectors) and personal information require access controls; consider encryption in production.

4. AI Algorithms

4.1. AI Pipeline Overview

The face recognition pipeline is: face detection → cropping and preprocessing → encoding into feature vectors → vector storage → similarity search → returning the match result and confidence score.

4.2. Face Detection (YOLOv8n)

- Principle: YOLO predicts bounding boxes and confidence scores for input images; non-maximum suppression (NMS) removes overlapping detections.
- Typical input: RGB images resized to a standard size (e.g., 640×640) to optimize speed.
- Output: face bounding-box coordinates and confidence scores; filter using a threshold (e.g., ≥ 0.5) to reduce noise.
- Optimization: Adjust resolution (320/640/1280) according to speed and accuracy requirements; batch processing improves throughput.

4.3. Face Encoding (FaceNet)

- Objective: Convert face images into 512-dimensional embeddings representing facial features.
- Preprocessing: Resize to 160×160 and normalize pixel values according to FaceNet standards (range [-1, 1]).
- Vector normalization: Apply L2 normalization before matching for more stable cosine similarity.
- Result: A 512-dimensional embedding used for storage and matching.

4.4. Matching and Similarity Search (Qdrant + Cosine)

- Cosine similarity measures the similarity of two vectors; for L2-normalized vectors, cosine similarity equals their dot product.
- Qdrant stores vectors and retrieves the top-k nearest vectors by cosine distance; score_threshold filters out weak matches.
- Match decision: If the score exceeds the configured threshold (e.g., 0.8), return the corresponding user and confidence score; otherwise return “not found.”

4.5. Performance and Reliability Optimization (Simplified)

- Model optimization: Export or quantize models (INT8/FP16), or use ONNX/TensorRT for GPU-based production deployments.
- Processing optimization: Batch encoding/search, limit faces processed per image, and cache repeated queries when appropriate.
- Error handling: Provide fallbacks for detection/encoding/search failures (reprocess images, lower thresholds, or return clear errors).

4.6. Quality Evaluation and Metrics

- Image/embedding quality: Assess brightness, contrast, and sharpness; reject low-quality images to reduce errors.
- Common metrics: accuracy, FAR/FRR, EER (especially for similarity threshold selection), and processing-time statistics.

Conclusion

The YOLOv8n + FaceNet + Qdrant pipeline enables real-time, scalable facial recognition, with adjustable thresholds and performance settings for different deployment environments.

5. API Documentation

5.1. Basic Information

Base URL: http://localhost:8000

Content-Type: application/json

API documentation: /docs (Swagger UI) and /redoc

Version: 1.0.0

5.2. General Conventions

REST API methods: GET (retrieve), POST (create/search), PUT (update), DELETE (remove).

Input validation uses Pydantic schemas.

CORS can be configured for applications hosted on other domains.

Authentication is not currently enabled; JWT may be added later.

5.3. Health Check

GET /health

- Purpose: Check system health and MongoDB/Qdrant/AI service status.

- Returns overall status (healthy/degraded/unhealthy) and system information (CPU/RAM/uptime).
- Response codes: 200 if healthy; 503 if one or more services are unavailable.

5.4. User Management

GET /users

- Retrieve users and their face-image thumbnails.
- Response code: 200; system error: 500.
- 4.2 POST /users
- Create a new user and register face images.
- Body: name, email, optional phone and department, images: 1–5 base64-encoded images.
- Response codes: 201; invalid data: 400/422; duplicate email: 409; oversized image: 413; system error: 500.

PUT /users/{user_id}

- Update user information (images are not updated through this endpoint).
- Response codes: 200; not found: 404; invalid data: 400/422; system error: 500.

DELETE /users/{user_id}

- Delete a user from the system, including related images and face vectors.
- Response codes: 204; not found: 404; system error: 500.

POST /users/{user_id}/images

- Add face images for an existing user (the total must not exceed five).
- Response codes: 200; not found: 404; oversized image: 413; invalid data: 400/422; system error: 500.

5.5. Face Recognition

POST /search

- Purpose: Find a user using a face image.
- Body: image (base64), optionally with threshold and max_results if supported.
- Result: match_found (true/false), user_info (if available), confidence/similarity_score, and processing information.
- Response codes: 200 (with or without a match); invalid image/no face: 400; oversized image: 413; validation error: 422; system error: 500; AI service unavailable: 503.

5.6. Limits and Recommendations

- Request size limit: approximately 50 MB; maximum per image: approximately 10 MB.
- For mobile requests, resize images before base64 encoding to reduce upload time and avoid 413 errors.
- If a timeout occurs, verify that the server and MongoDB/Qdrant containers are running.

5.7. Common HTTP Status Codes

200: Success

201: Created successfully

204: Deleted successfully (no response body)

400: Invalid data

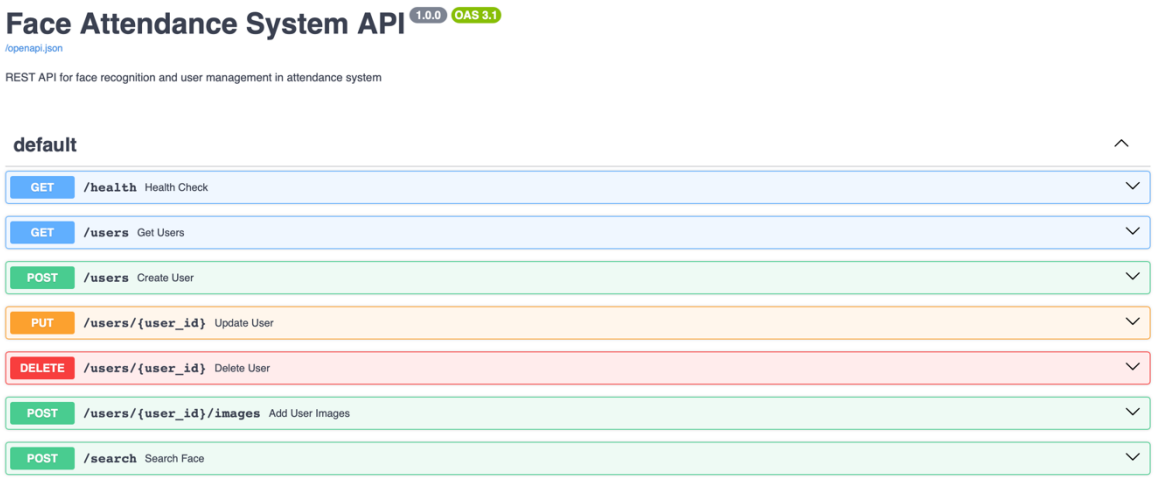
404: Not found

422: Validation error

500: System error

503: Service temporarily unavailable

Note: This summary is shortened to fit 1–2 report pages.



6. Database

6.1. Database Architecture Overview

- MongoDB: Stores user data and metadata as documents.
- Qdrant: Stores face embeddings and supports similarity search.
- The two systems have separate roles: MongoDB handles business data, while Qdrant is optimized for vector matching.

6.2. MongoDB

- Connection port: 27018. Database: face_attendance.
- Main collection: users (user information and face-image list).
- Optional collections: system_config (threshold settings), logs/audit (access tracking), attendance_logs (for future attendance features).

6.2.1. Users Data Structure (Simplified)

- Basic information: id, name, email, phone, department, status, created_at, updated_at.
- images: Face-image list (maximum five per user). Each image has image_id, full_image (base64), thumbnail (base64), face_vector_id (Qdrant reference).
- metadata (optional): total_images, recognition_count, avg_confidence, last_recognition, etc., for statistics.

6.2.2. Suggested Indexes and Constraints

- Unique index on email to prevent duplicate users.
- Indexes on name/department/status/created_at to speed up listing and filtering.
- Text index on name/email/department to support keyword searches.

6.3. Qdrant (Vector Database)

- REST port: 6333; gRPC: 6334.
- Main collection: face_vectors. Vector dimensions: 512. Distance: Cosine.

6.3.1. Structure of a Vector

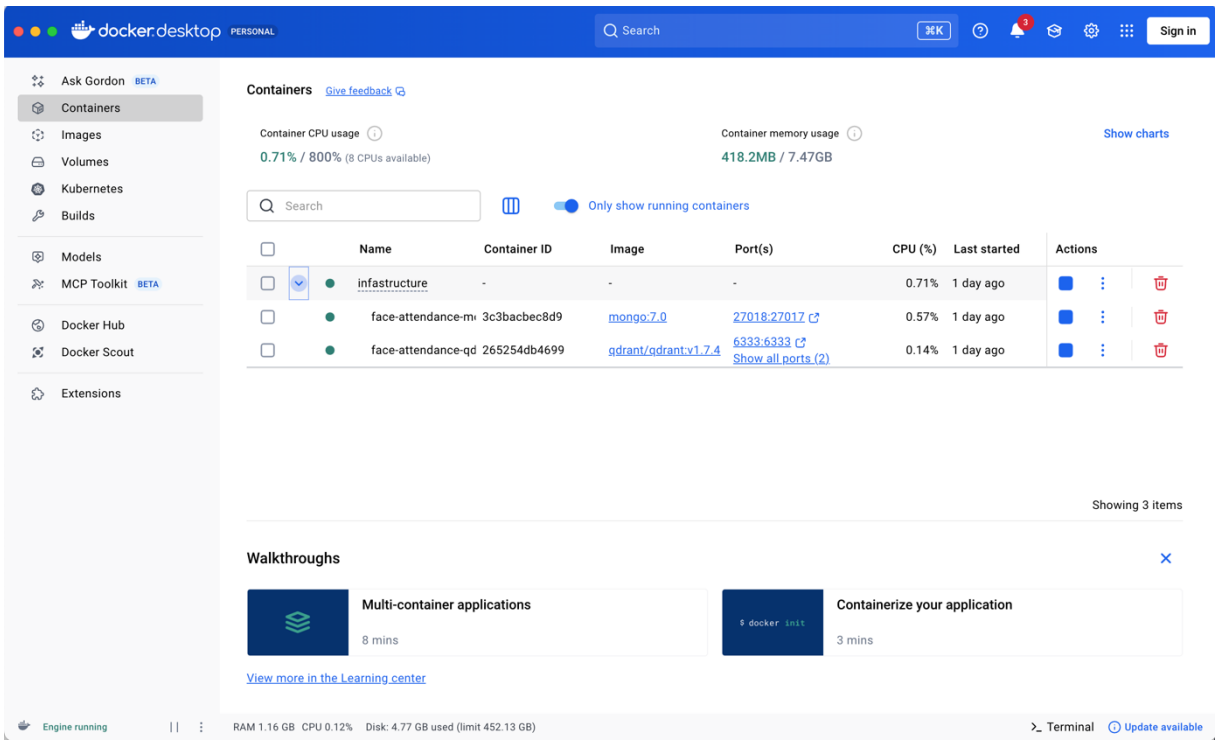
- id (UUID): Vector identifier.
- vector: List of floating-point values (embedding) representing a face.
- payload: user_id, image_id, creation time, bbox/confidence information, and optional tags/department.

6.4. Storage Flow and Data Integrity

- When creating a user: process images → generate embeddings → store embeddings in Qdrant → store user information and images/thumbnails in MongoDB.
- When searching: generate an embedding from the query image → Qdrant returns the top-k similar vectors → retrieve user information from MongoDB.
- When deleting a user: delete user/images in MongoDB and all related vectors in Qdrant to avoid orphaned data.
- Recommendation: Normalize embeddings before storage and search to maintain stable cosine scores.

6.5. Operations

- Backup: Use scheduled mongodump backups for MongoDB and scheduled collection snapshots for Qdrant.
- Monitoring: Track storage, vector counts, query speed, and container status.
- Security: Enable MongoDB authentication/authorization, restrict bindIp, and control access to biometric data (face vectors).



7. User Interface

7.1. Interface Objectives

- Display users who have registered their faces.
- Allow users to be added with face images selected from the photo library.
- Allow searching for users using face images selected from the photo library.
- Clearly display search results (match/no match) and user information.

7.2. Overall User Flow

- Initial screen: User List.
- From the User List, tap “Add” to open the Add User screen.
- From the User List, tap “Search by Face” to select an image and view search results.

7.3. User List Screen

- Purpose: Quickly display system users and serve as the main navigation point.
- UI components:
- Navigation bar: “Users” title.
- “+” (Add) button: Navigates to the Add User screen.
- “Search by Face” button: Opens the image selection and search flow.
- List: Each row displays Name and Email.
- Display states:
- Loading: Show a spinner or placeholder while data is loading.
- Empty: Show a brief message and suggest tapping “+” to add a user.
- Error: Show an alert and a Retry button.

7.4. Add User Screen

- Purpose: Enter basic information and select a face image to register a user.
- UI components:
- Name input field.
- Email input field.
- “Pick Image” button: Opens the photo library to select one image.
- Selected-image preview (thumbnail) for user confirmation.
- “Submit” button: Sends the information; on success, returns to and refreshes the User List.
- Suggested validation rules:
- Name and Email must not be empty.
- Email must have a valid basic format.
- An image must be selected before submission.
- Display states:
- Submitting: Disable Submit and show progress to prevent duplicate submissions.
- Failure: Display an error and retain entered information for easy correction.

7.5. Search by Face Screen

- Purpose: Let the user select a face image to perform a search.
- Recommended minimal design:
- A separate screen may not be necessary: tapping “Search by Face” on the User List can open the photo library directly.
- After image selection, show “Searching…” and navigate to the Results screen.
- Display states:
- If image selection is canceled, return to the User List without changing data.

7.6. Search Result Screen

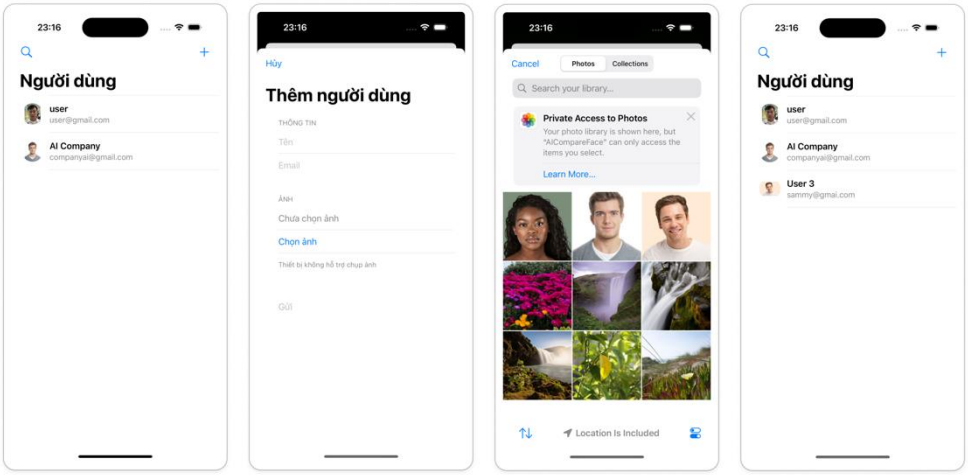
- Purpose: Present recognition results clearly and understandably.
- When a match is found:
- Display Name, Email, and optional Phone and Department fields.
- Display Confidence as a percentage or score.
- “Done/Back” button returns to the User List.
- When no match is found:
- Display “No matching user found.”
- Allow “Try Again” to select another image from the photo library.

7.7. User Experience Notes

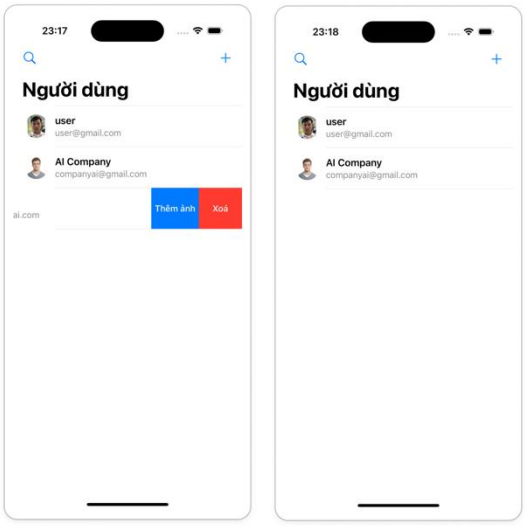
- Preview selected images so users can confirm their choice.
- For large images, automatically reduce their dimensions before processing to improve responsiveness.
- Keep error messages concise and focused on the next action (retry/select another image).
- Buttons and text should be large enough and provide basic Dynamic Type and VoiceOver support.

7.8. Product Information

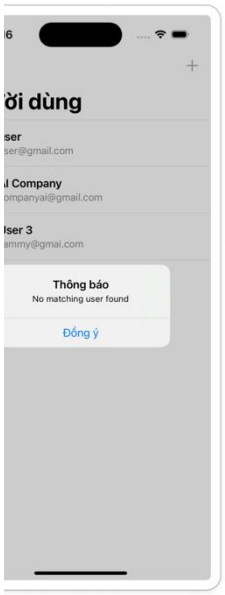
THÊM NGƯỜI DÙNG



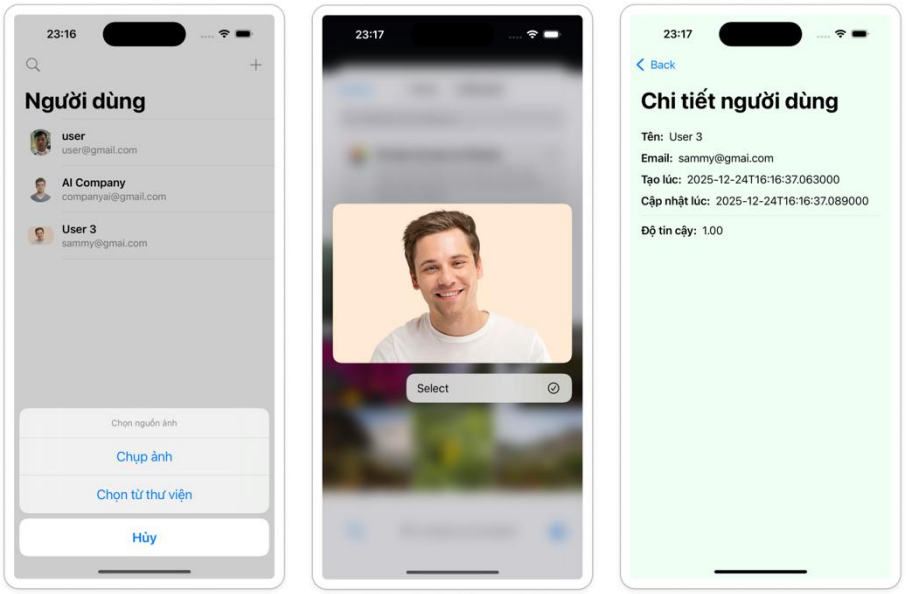
XOÁ NGƯỜI DÙNG



BẠI



TÌM KIẾM NGƯỜI DÙNG THÀNH CÔNG



8. Appendix

Knowledge bases and technology platforms used in the application:

- Artificial Intelligence and Computer Vision (Computer Vision, Deep Learning)
- YOLOv8 (Ultralytics) model for face detection
- FaceNet (InceptionResnetV1) model for facial feature vector extraction
- MongoDB database for user data storage
- Qdrant vector database for similarity search
- Layered Architecture combined with Microservices
- FastAPI backend – RESTful API
- SwiftUI – Swift – iOS frontend
- Docker, Docker Compose, and Kubernetes deployment (planned)

9. References

- Ultralytics YOLOv8 Documentation: https://docs.ultralytics.com/
- FaceNet Paper – Schroff et al.: https://arxiv.org/abs/1503.03832
- PyTorch Documentation: https://pytorch.org/docs/stable/index.html
- FastAPI Documentation: https://fastapi.tiangolo.com/
- MongoDB Documentation: https://www.mongodb.com/docs/
- Qdrant Vector Database Documentation: https://qdrant.tech/documentation/
- Apple SwiftUI Documentation: https://developer.apple.com/documentation/swiftui/
- Docker Documentation: https://docs.docker.com/
- Kubernetes Documentation: https://kubernetes.io/docs/